



OASIS Security Report

Sensitive Data Exposure Security Analysis

Date: 2026-06-11 23:44:34 — **Model:** llama3.2:3b

Vulnerability: Sensitive Data Exposure

Project: vulnerable

Analysis root: `../vulnerable`

Table of contents

□ Summary

□ Statistics

□ Files

□ Detailed analysis

□ Assistant

△ Errors & notes

Summary

Analyzed **11** file(s) for Sensitive Data Exposure.

[Return to table of contents](#)

Statistics

Files analyzed: 11 **Total findings:** 18



Medium

5

Low

3

[Return to table of contents](#)

Files

File	Similarity
/work/project/dataset/vulnerable/oracle-manipulation/PuppetPool.sol	0.683
/work/project/dataset/vulnerable/oracle-manipulation/PuppetV2Pool.sol	0.675
/work/project/dataset/vulnerable/reentrancy/ 0xbaf51e761510c1a11bf48dd87c0307ac8a8c8a4f.sol	0.655
/work/project/dataset/vulnerable/access-control/mycontract.sol	0.655
/work/project/dataset/vulnerable/reentrancy/etherstore.sol	0.655
/work/project/dataset/vulnerable/ordering-attacks/FindThisHash.sol	0.643
/work/project/dataset/vulnerable/reentrancy/simple_dao.sol	0.632
/work/project/dataset/vulnerable/ordering-attacks/ eth_tx_order_dependence_minimal.sol	0.620
/work/project/dataset/vulnerable/access-control/parity_wallet_bug_1.sol	0.599
/work/project/dataset/vulnerable/ordering-attacks/odds_and_evens.sol	0.587
/work/project/dataset/vulnerable/access-control/parity_wallet_bug_2.sol	0.581

[Return to table of contents](#)

Detailed analysis

File 1: /work/project/dataset/vulnerable/oracle-manipulation/PuppetPool.sol

Similarity score: 0.683

Source lines: 1-71

► Finding 1.1: API Key Hardcoded (Medium)

Vulnerable code

```
function borrow(uint256 amount, address recipient) external payable
nonReentrant {
    ...
    unchecked {
        deposits[msg.sender] += depositRequired;
    }

    // Fails if the pool doesn't have enough tokens in liquidity
    if (!token.transfer(recipient, amount)) {
        revert TransferFailed();
    }

    emit Borrowed(msg.sender, recipient, depositRequired, amount);
}
```

Explanation

The `token` object's address is hardcoded as `DamnValuableToken(tokenAddress)`, making it vulnerable to exposure.

Impact

Unauthorized access to the token balance or transfer functions.

Entry point

/borrow(uint256 amount, address recipient)

HTTP methods

POST

Exploitation steps

- Get the hardcoded `token` object's address.

Conditions

Hardcoded API key is used.

Remediation

Use environment variables or secure storage solutions for sensitive data.

Secure example

```
const tokenAddress = process.env.TOKEN_ADDRESS;  
DamnValuableToken public immutable token;
```

File 2: /work/project/dataset/vulnerable/oracle-manipulation/PuppetV2Pool.sol

Similarity score: 0.675

Source lines: 1-70

► Finding 2.1: Hardcoded Secret (High)

Vulnerable code

```
_weth = IERC20(wethAddress); _token = IERC20(tokenAddress);
```

Explanation

The addresses of WETH and token are hardcoded, making it vulnerable to exposure.

Impact

Unauthorized access to the contract's functionality

Entry point

/constructor/

Execution path

```
````WETH_ADDRESS = _wethAddress; TOKEN_ADDRESS = _tokenAddress;````  

_contract().constructor(WETH_ADDRESS, TOKEN_ADDRESS)
```

## File 3: /work/project/dataset/vulnerable/reentrancy/0xbaf51e761510c1a11bf48dd87c0307ac8a8c8a4f.sol

Similarity score: 0.655

Source lines: 1-85

### ► Finding 3.1: Hardcoded Secret (Medium)

#### Vulnerable code

```
`pragma solidity ^0.4.19;`
```

#### Explanation

The hardcoded contract version could allow an attacker to target specific vulnerabilities.

#### Impact

Potential exposure of private contract data.

#### Entry point

/main

#### Exploitation steps

- Recover hardcoded contract version

#### Example payloads

- Old contract version

#### Remediation

Update the contract to use environment variables for sensitive data.

#### Secure example

```
pragma solidity ^<version>;
contract ETH_VAULT ...
```

## File 4: /work/project/dataset/vulnerable/access-control/mycontract.sol

Similarity score: 0.655

Source lines: 1-31

### ► Finding 4.1: Hardcoded Secret (Medium)

#### Vulnerable code

```
address owner = msg.sender;
receiver.transfer(amount);
require(tx.origin == owner);
```

#### Explanation

The `owner` variable is hardcoded and can be accessed directly, allowing for potential manipulation.

#### Impact

Unauthorized access to the contract's state

#### Entry point

MyContract() public function MyContract() public function sendTo(address receiver, uint amount) public

#### Parameters

owner

#### Exploitation steps

- An attacker can access the owner address and perform actions on behalf of the contract

#### Conditions

No access controls are in place to restrict tx.origin

#### Remediation

Use a secure storage solution, such as an externally owned account or a modifier function.

## File 5: /work/project/dataset/vulnerable/reentrancy/etherstore.sol

Similarity score: 0.655

Source lines: 1-39

### ► Finding 5.1: Plaintext Credentials (High)

#### Vulnerable code

```
function depositFunds() public payable {
 balances[msg.sender] += msg.value;
}
```

#### Explanation

The contract stores user balances in a mapping, exposing them to unauthorized access.

#### Impact

Potential for theft of funds or sensitive information.

#### Entry point

/depositFunds

#### HTTP methods

POST

#### Parameters

msg.value, msg.sender

#### Exploitation steps

- Send a large value to overflow the user's balance.

#### Example payloads

- Large amount of ether

#### Conditions

User has sufficient Ether.

#### Remediation

Store balances in encrypted form using secure storage solutions like Solidity's `keccak256` function.

### Secure example

```
balances[msg.sender] = keccak256(balances[msg.sender].toString());
```

## File 6: /work/project/dataset/vulnerable/ordering-attacks/FindThisHash.sol

Similarity score: 0.643

Source lines: 1-28

### ► Finding 6.1: Hardcoded Secret (High)

#### Vulnerable code

```
bytes32 constant public hash =
0xb5b97fafd9855eec9b41f74dfb6c38f5951141f9a3ecd7f44d5479b630ee0a;
```

#### Explanation

The 'hash' variable is hardcoded with a plaintext value, making it vulnerable to exposure.

#### Impact

An attacker can access the hardcoded hash value.

#### Entry point

/contract/FindThisHash/solve

#### Parameters

solution

#### Exploitation steps

- Get the value of 'hash' from the contract

#### Conditions

Contract is deployed and accessible.

#### Remediation

Replace hardcoded hash with a secure storage solution.

## File 7: /work/project/dataset/vulnerable/reentrancy/simple\_dao.sol

Similarity score: 0.632

Source lines: 1-35

### ► Finding 7.1: Hardcoded API Key (Low)

#### Vulnerable code

```
pragma solidity ^0.4.2;
contract SimpleDAO {
 mapping (address => uint) public credit;
 function donate(address to) payable {
 credit[to] += msg.value;
 }
 function withdraw(uint amount) {
 if (credit[msg.sender]>= amount) {
 bool res = msg.sender.call.value(amount)();
 credit[msg.sender]-=amount;
 }
 }
 function queryCredit(address to) returns (uint){
 return credit[to];
 }
}
```

#### Explanation

The API key is hardcoded as `pragma solidity ^0.4.2;` which can be easily accessed and used by an attacker.

#### Impact

An attacker could use the hardcoded API key to make unauthorized transactions.

#### Entry point

/withdraw(uint)

#### Parameters

amount

#### Exploitation steps

- Attackers can exploit the hardcoded API key by making a withdrawal request.

#### Example payloads

- `invalid payload`

## Conditions

API key is hardcoded in the contract.

## Remediation

The API key should be stored securely, such as using an environment variable or a secrets manager.

## Secure example

```
pragma solidity ^0.8.0;
contract SimpleDAO {
 mapping (address => uint) public credit;
 function donate(address to) payable {
 credit[to] += msg.value;
 }
 function withdraw(uint amount) {
 if (credit[msg.sender] >= amount) {
 bool res = msg.sender.call.value(amount)();
 credit[msg.sender] -= amount;
 }
 }
 function queryCredit(address to) returns (uint){
 return credit[to];
 }
}
```

## File 8: /work/project/dataset/vulnerable/ordering-attacks/eth\_tx\_order\_dependence\_minimal.sol

Similarity score: 0.620

Source lines: 1-42

### ► Finding 8.1: Hardcoded API Key (High)

#### Vulnerable code

```
address public owner;
bool public claimed;
uint public reward;

function EthTxOrderDependenceMinimal() public {
 owner = msg.sender;
}
function setReward() public payable {
 require (!claimed);

 require(msg.sender == owner);
 owner.transfer(reward);
 reward = msg.value;
}
```

#### Explanation

The contract hardcodes the `owner` variable, making it accessible to anyone who can read or modify the contract.

#### Impact

An attacker could modify the `owner` variable to gain unauthorized access to the contract.

#### Entry point

setReward() function

#### Execution path

Owner -> Set Reward -> Transfer -> Reward

#### Parameters

owner

#### Exploitation steps

- Modify the `owner` variable to gain access to the contract.

## Conditions

Owner is set before execution.

## Remediation

Store the owner address in a secure storage mechanism, such as a private key or a trusted dependency.

## Secure example

```
address public owner;
address privateOwnerAddress = 0x...;
function setReward() public payable {
 require (!claimed);
 owner = privateOwnerAddress;
 owner.transfer(reward);
 reward = msg.value;
}
```

## File 9: /work/project/dataset/vulnerable/access-control/parity\_wallet\_bug\_1.sol

Similarity score: 0.599

Source lines: 1-225

### ► Finding 9.1: Hardcoded API key (High)

#### Vulnerable code

```
pragma solidity ^0.4.9; contract WalletEvents {
 function initMultiowned(address[] _owners, uint _required) {
 // ... hardcoded api key here ...
 }
}
```

#### Explanation

The `initMultiowned` function contains a hardcoded API key.

#### Impact

Exposure of sensitive data to unauthorized parties.

#### Entry point

/contract/WalletEvents/initMultiowned(address[], uint)

#### Parameters

\_owners, \_required

#### Exploitation steps

- Get the address array and API key.

Source lines: 1-281

### ► Finding 9.1: Hardcoded Secret (Low)

#### Vulnerable code

```
pragma solidity 0.4.9; //sol Wallet
contract WalletEvents {
 ...
}
```

#### Explanation

The wallet contract contains a hardcoded secret in the `pragma solidity` directive.

**Impact**

An attacker with access to the contract's source code could potentially extract the hardcoded secret.

**Entry point**

/pragma solidity 0.4.9;

**Exploitation steps**

- Read the contract's source code.

**Remediation**

Use a secure configuration file or environment variable to store sensitive data.

**Source lines:** 1-320

► **Finding 9.1: Hardcoded secret (Medium)**

**Vulnerable code**

```
function initDaylimit(uint _limit) {
 m_dailyLimit = _limit;
 m_lastDay = today();
}
```

**Explanation**

The daily limit is hardcoded, making it vulnerable to exposure.

**Impact**

An attacker could potentially modify the daily limit.

**Entry point**

initDaylimit(uint \_limit)

**Execution path**

```
DailyLimit -> initDaylimit() -> setDailyLimit(uint _newLimit) -> m_dailyLimit = _limit;
```

**Parameters**

\_limit

**Exploitation steps**

- Modify the daily limit value

**Example payloads**

- ```
uint256 newLimit = 1000000;
setDailyLimit(newLimit);
```

Conditions

The user has administrative privileges.

Remediation

Use environment variables or secure storage for sensitive data.

Secure example

```
import os
m_dailyLimit = os.getenv('DAILY_LIMIT')
m_lastDay = today()
```

Source lines: 27-358

► **Finding 9.1: Hardcoded API Keys (Medium)**

Vulnerable code

```
function revoke(bytes32 _operation) external {
    // make sure they're an owner
    if (ownerIndex == 0) return;
    uint ownerIndexBit = 2**ownerIndex;
    var pending = m_pending[_operation];
    if (pending.ownersDone & ownerIndexBit > 0) {
        pending.yetNeeded++;
        pending.ownersDone -= ownerIndexBit;
        Revoke(msg.sender, _operation);
    }
}
```

Explanation

The revoke function does not check for hardcoded API keys or sensitive data exposure

Impact

Exposure of confidential information through hardcoded API keys

Entry point

revoke(bytes32)

Parameters

_operation

Exploitation steps

- Send a revoke request with the correct operation hash

Conditions

Hardcoded API keys are not validated or secured

Remediation

Use secure storage solutions for sensitive data and validate API keys

Secure example

```
function revoke(bytes32 _operation) external {
    // use secure storage for sensitive data
    uint ownerIndex = m_ownerIndex[uint(msg.sender)];
    var pending = m_pending[_operation];
    if (pending.ownersDone & ownerIndex > 0) {
        pending.yetNeeded++;
        pending.ownersDone -= ownerIndex;
        Revoke(msg.sender, _operation);
    }
}
```

Source lines: 126-467

► Finding 9.1: Hardcoded API Key (Critical)

Vulnerable code

```
"_walletLibrary" = 0xafecafecafecafecafecafecafecafecafe;
```

Explanation

The wallet library's address is hardcoded, making it accessible to anyone who can view the code.

Impact

An attacker with access to the contract's source code can exploit this vulnerability.

Entry point

```
initWallet(address[] _owners, uint _required, uint _daylimit)
```

Execution path

```
Hardcoded API Key -> initWallet() -> _walletLibrary = ...
```

Exploitation steps

- Read the hardcoded API key from the contract's source code

Example payloads

- 0xafecafecafecafecafecafecafecafecafe

Conditions

Contract source code is publicly accessible.

Remediation

Replace hardcoded API key with an environment variable or secure storage solution.

Secure example

```
const _walletLibrary = process.env.WALLET_LIBRARY_ADDRESS || '...';
```

Source lines: 207-477

► Finding 9.1: Hardcoded API Key (High)

Vulnerable code

```
function initWallet(address[] _owners, uint _required, uint _daylimit) { ...
function setDailyLimit(uint _newLimit) onlymanyowners(sha3(msg.data)) external
{ m_dailyLimit = _newLimit; ... } ... }
```

Explanation

The contract uses hardcoded API keys and sensitive data without proper protection.

Impact

Exposure of confidential information, such as API keys.

Entry point

/initWallet(address[],uint256,uint256)

Parameters

_owners, _required, _daylimit

Exploitation steps

- Obtain the hardcoded API key and sensitive data.

Remediation

Use secure storage solutions, such as encrypted variables or environment variables.

Secure example

```
const uint _api_key = 0x1234567890abcdef;
const address[] _owners = [...];
contract Wallet(...) { ... }
```

File 10: /work/project/dataset/vulnerable/ordering-attacks/odds_and_evens.sol

Similarity score: 0.587

Source lines: 1-61

► Finding 10.1: Plaintext Credentials (Critical)

Vulnerable code

```
function OddsAndEvens() {  
  owner = msg.sender;  
}
```

Explanation

The owner address is stored in plaintext, making it vulnerable to exposure.

Impact

Exposure of the owner's Ethereum wallet address.

Entry point

/contract/owner

Execution path

```
msg.sender -> owner = msg.sender  
owner (plaintext) -> contract execution
```

Parameters

msg.sender

Exploitation steps

- Obtain the owner's Ethereum wallet address

Conditions

Authenticated sender can read the owner's address.

Remediation

Use a secure storage solution for sensitive data, such as Keystore or Safe

Secure example

```
owner = keystore.loadAddress(msg.sender);
```

File 11: /work/project/dataset/vulnerable/access-control/parity_wallet_bug_2.sol

Similarity score: 0.581

Source lines: 1-227

► Finding 11.1: Hardcoded API Key (High)

Vulnerable code

```
pragma solidity ^0.4.9;
contract WalletAbi {
    function execute(address _to, uint _value, bytes _data) external returns
    (bytes32 o_hash) { return sha3(msg.sender);
    }
```

Explanation

The "execute" function uses the "msg.sender" variable to compute the operation hash without any input validation.

Impact

An attacker can access the wallet's functionality using hardcoded API keys.

Entry point

/execute

HTTP methods

POST

Parameters

_to, _value, _data

Exploitation steps

- Obtain a valid msg.sender address

Example payloads

- `https://example.com/execute?_to=0x12345678901234567890123456789012&_value=100&_data="\n"`

HTTP raw requests (Burp / ZAP)

```
POST /execute HTTP/1.1
Host: example.com
Content-Type: application/json
```

```
{"_to":"0x12345678901234567890123456789012","_value":100,"_data":"\\n"}
```

Source lines: 1-286

► Finding 11.1: Hardcoded Secret (Low)

Vulnerable code

```
function initDaylimit(uint _limit) only_uninitialized { m_dailyLimit = _limit;  
m_lastDay = today(); }
```

Explanation

The `initDaylimit` function directly uses the `_limit` parameter, making it a hardcoded secret.

Impact

An attacker could modify this value to manipulate the daily limit.

Entry point

`/contract/WalletLibrary/initDaylimit(uint _limit)`

Parameters

`_limit`

Exploitation steps

- Call `initDaylimit` with a malicious `_limit` value

Conditions

Admin has administrative access to the contract.

Remediation

Use a secure storage mechanism for the daily limit value.

Source lines: 96-414

► Finding 11.1: Unencrypted Sensitive Data (High)

Vulnerable code

```
uint public m_spentToday;
```

Explanation

The `m\_spentToday` variable is not encrypted, making it vulnerable to exposure.

Impact

Exposure of daily spending limit could be used for unauthorized transactions.

Entry point

execute(address _to, uint _value, bytes _data)

Execution path

```
execute -> m_dailyLimit -> m_spentToday;
```

Parameters

m_dailyLimit, m_spentToday

Exploitation steps

- Obtain access to the contract's storage.
- Read `m\_spentToday` variable.

HTTP raw requests (Burp / ZAP)

```
GET /execute HTTP/1.1
```

```
Host: <contract_address>
```

[Return to table of contents](#)

Assistant (triage / codebase)

In the OASIS dashboard (JSON report preview), use the chat below to triage findings, exploitation paths, and remediation. Select file, chunk, and finding to narrow context when needed.

[Return to table of contents](#)

Errors & Notes

No file-level errors were recorded.

[Return to table of contents](#)